

The Memory Wall

Why the $\Omega=100$ case study won't run yet on this machine, what four attempts at it actually showed, and what closing the gap would take.

Prepared 2026-09-14 Session work 2026-09-09 Machine MacBook Pro • Apple M3 Pro • 18 GB
Solver Gurobi 11.0.3

18 GB total system memory	4 / 4 $\Omega=100$ attempts, none completed	65 s root relaxation solve, reproduced twice	0 MIP incumbents found so far
---	--	--	---

Bottom line

PRS-VIF's structural formulation checks out at every scale actually tested — the real 50-node, 96-vehicle network builds and its LP relaxation solves cleanly through $\Omega=50$ scenarios, matching every dimension the dissertation's case study specifies. The target scale, **$\Omega=100$, does not:** four independent attempts to build or solve it have now failed, all tracing to the same underlying cause — this laptop's 18 GB of RAM, not a defect in the model or the code that builds it.

A first real MIP attempt at a reduced but still CVaR-meaningful scale ($\Omega=30$, $\beta=0.8$) got further than anything before it — a clean, twice-reproduced root-relaxation solve — before running out of memory during Gurobi's crossover step, *before branch-and-bound ever started*. Nothing tried so far (bulk constraint construction, fewer solver threads, disk-backed node storage) has closed the gap. The two paths that plausibly would are more RAM, or a smaller and better-conditioned formulation.

Why $\Omega=100$ isn't just a nice round number

CVaR's tail size is $(1 - \beta) \times N$. At the dissertation's calibrated $\beta=0.9$, that arithmetic is unforgiving: $N=100$ gives a tail of 10 scenarios — enough for the risk measure to actually *average* a set of bad outcomes. $N=10$ gives a tail of exactly 1 — CVaR stops being a conditional average and collapses to a single worst-case draw, a degeneracy already observed by accident in earlier unit testing. $N=50$ gives a tail of 5: non-degenerate, but thin.

So reducing the scenario count isn't a free way around the memory ceiling — it changes what the objective means. The one deliberate exception tried here is $\beta=0.8$, $N=30$ (tail = 6, comparable richness to $N=50$ at $\beta=0.9$), chosen specifically to stay inside the memory-safe range while keeping CVaR non-degenerate. See §4.

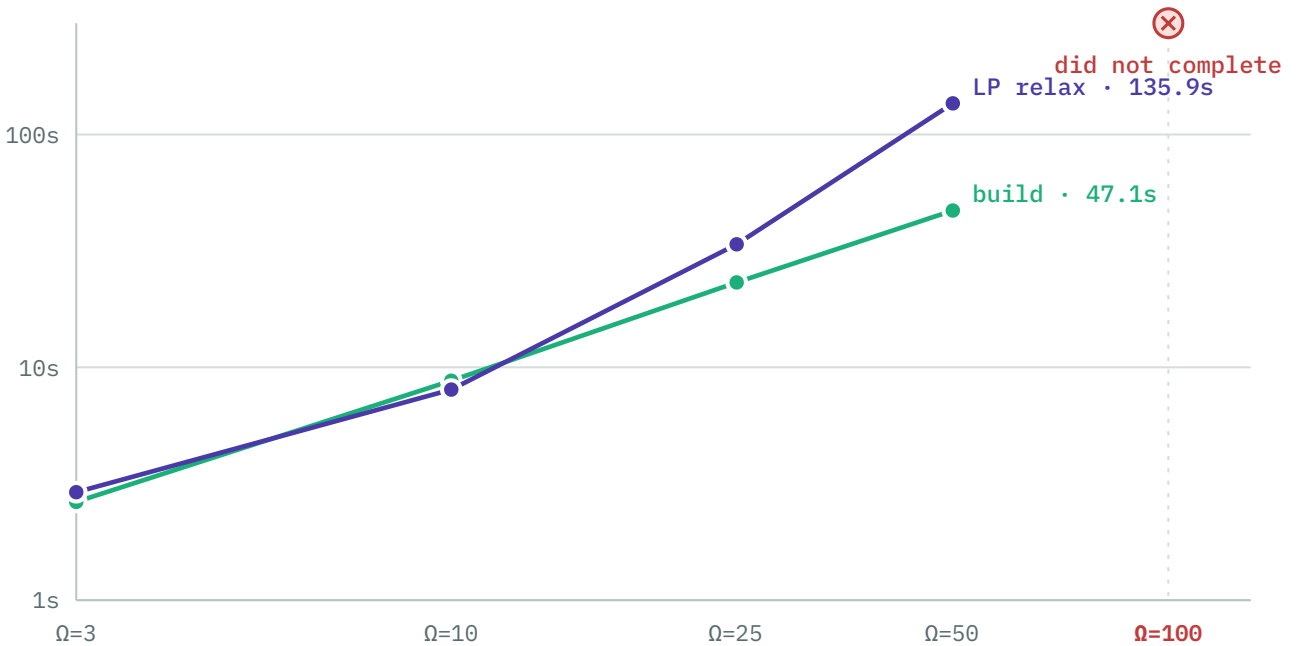
Structural scaling, $\Omega = 3$ to 50

Every instance below is the real 50-node USPACOM network (22 PPL candidates, 1,502 air / 236 sea / 30 land arcs) with the full 96-vehicle fleet — not a toy or reduced case. Each was built and LP-relaxed to confirm the model assemblies correctly at scale before any integer solve was attempted.

Build and LP-relaxation timing, real network, by scenario count Ω						
Ω	BUILD (S)	LP RELAX (S)	VARIABLES	BINARY VARS	CONSTRAINTS	LP OBJECTIVE
3	2.64	2.91	428,189	138,718	154,238	5,074,641,919.53
10	8.75	8.02	1,423,827	458,926	510,482	—
25	23.14	33.72	3,557,337	1,145,086	1,273,862	10,679,050,348.59
50	47.05	135.86	7,113,187	2,288,686	2,546,162	11,536,060,439.90

Build time vs. LP-relaxation time, log-log

One trend is a straight line. The other curves away from it.



— build time — linear (exponent ≈ 1.0 – 1.06)

— LP-relax time — accelerating (exponent $0.84 \rightarrow 1.57 \rightarrow 2.01$)

Extrapolating the reliable linear build trend alone predicts an $\Omega=100$ build of roughly 100 seconds. That prediction was tested directly — see §3.

Four attempts at $\Omega=100$

All four used the real network at full scale ($\approx 14.2\text{M}$ variables, $\approx 5.1\text{M}$ constraints, extrapolated). None reached a completed build. Two hit a genuine multi-hour resource thrash; one was a false alarm from an over-cautious safety script; one hit a real ceiling mid-growth.

Attempt 1 — unmonitored background run

THRASHED

Ran 80+ minutes of CPU time without ever reaching a completed build. Process memory stayed flat at 2.8–3.5 GB the entire time — no forward progress, not the healthy growth curve seen at every smaller Ω . System swap climbed to 4.1 of 5.0 GB used; free system RAM fell to 0.06 GB. Terminated by the operating system for system-wide low memory.

duration **80+ min** peak RSS **~ 3.5 GB (flat)** outcome **OS-killed**

Attempt 2 — same code, actively monitored

THRASHED

Memory grew normally to ~ 7 GB in the first 90 seconds — matching the confirmed healthy pattern — then plateaued for **over three hours** with CPU pegged and zero further progress: the same thrash signature as Attempt 1, this time observed live rather than inferred after the fact. Self-terminated as a manual kill was being issued.

duration **3h 25m+** peak RSS **~ 7 GB, then flat ~ 3.5 GB** outcome **self-terminated**

Attempt 3 — after bulk-construction rewrite

FALSE ALARM

Run with the two largest constraint families converted to a single bulk Gurobi call each (§4). Killed by my own safety monitor after 30 seconds — a tooling false positive, not a model problem: RSS at that point (~ 7 GB) matched the same normal, healthy pattern every smaller Ω already showed. The monitor's threshold was miscalibrated against this machine's already-reduced baseline headroom.

duration **30 s** RSS at kill **~ 7 GB (normal)** outcome **false-positive kill**



Attempt 4 — corrected monitor (targets stalling, not a single reading)

GENUINE CEILING

Memory climbed steadily to 7.79 GB — real, continued growth, not a stall — when free system memory hit genuine near-zero and stayed there for two consecutive checks. The first of the four kills that reflects an actual resource ceiling rather than a monitoring error.

duration ~2 min RSS at kill 7.79 GB, still climbing

outcome deliberate kill, real limit

04

Did rewriting the construction code help?

The two constraint families that scale as $\Omega \times |\text{vehicles}| \times |\text{arcs}|$ — together the overwhelming majority of the model's rows — were originally built with one Python call per row (4.57M individual calls at $\Omega=100$). These were converted to Gurobi's bulk-insert API, which hands the solver the whole family in a single call.

Verified mathematically identical: rebuilt at $\Omega=3$, every variable and constraint count and the LP-relaxation objective (5,074,641,919.5340) matched the pre-conversion run to the decimal.

Made no measurable difference: at $\Omega=50$, build and relax times were statistically unchanged from the per-row version.

$\Omega=50$, before vs. after the bulk-construction rewrite			
VERSION	BUILD (S)	LP RELAX (S)	LP OBJECTIVE
Per-row addConstr()	47.05	135.86	11,536,060,439.90
Bulk addConstrs()	52.68	136.56	(re-validated at $\Omega=3$, identical)

Attempts 3 and 4 at $\Omega=100$ used this rewritten code and hit the same wall as Attempts 1 and 2. Ruled out as the fix.

First real MIP attempt — $\Omega=30$, $\beta=0.8$

Everything above only ever tested the LP relaxation. This is the first attempt to actually solve PRS-VIF with integrality enforced, at any scale, on the real network. Model size: 1,528,322 rows, 4,268,507 columns (2.89M continuous, 1.37M binary), 19,395,010 nonzeros.

Run 1 — default solver threads

12 threads · time limit 20 min

Presolve	11.44 s
----------	---------

Root barrier solve	58.11 s
--------------------	---------

Root objective	6.63358571×10^9
----------------	--------------------------

Crossover	completed, 65 s total
-----------	-----------------------

Killed at	~2 min, 7.6 GB RSS
-----------	--------------------

Reached B&B?	no — killed right as it began
--------------	-------------------------------

Run 2 — 4 threads, disk-backed node storage

Threads=4 · NodefileStart=4.0 GB · time limit 20 min

Presolve + barrier	50.27 s
--------------------	---------

Root objective	6.63358571×10^9 (identical to Run 1)
----------------	---

Crossover	unstable, did not finish
-----------	--------------------------

Dual infeasibility spike	7.46×10^{23}
--------------------------	-----------------------

Simplex iterations	350,000+, still running
--------------------	-------------------------

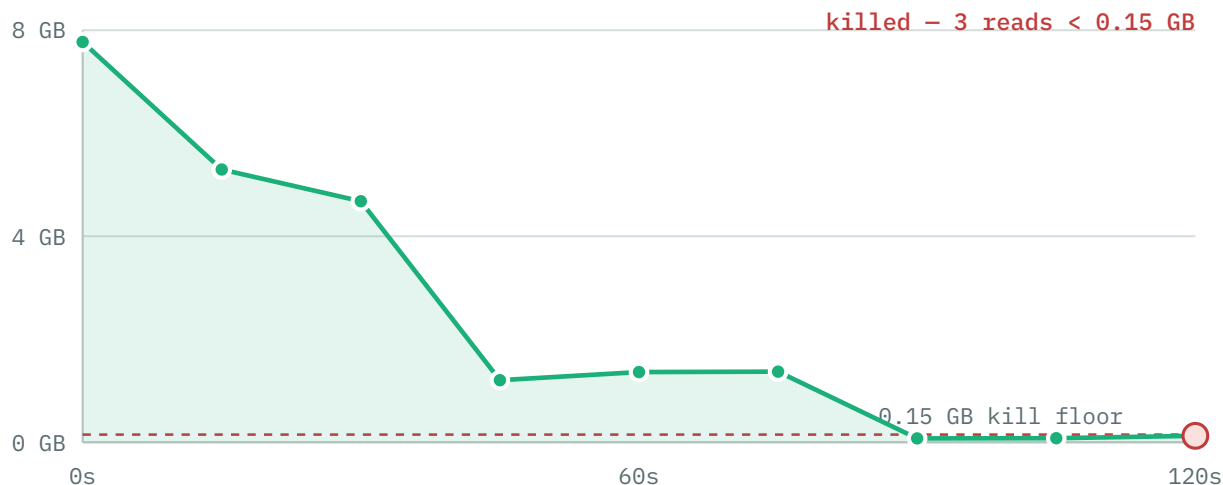
Killed at	~2 min, 8.1 GB RSS
-----------	--------------------

```
Root simplex log, Run 2 — the instability that ended the run: 302503 6.6335857e+09
0.000000e+00 7.608578e+03 55s 304964 7.4551148e+23 1.304443e+11 0.000000e+00 61s 305604
6.6335857e+09 3.164302e+11 0.000000e+00 66s Warning: Markowitz tolerance tightened to 0.5
```

Both runs reproduce the *identical* root-relaxation objective under different solver configurations — strong evidence the model itself is correct and deterministic. The obstacle both times is memory, at two different points: Run 1 ran out right as branch-and-bound was starting; Run 2's crossover went numerically unstable first and grew to 8.1 GB chasing it. **Neither run has ever reached branch-and-bound.**

Free system memory during Run 2

Sampled every 15 seconds, from launch to kill.



06

Diagnosis

- The formulation itself is sound: every scale from $\Omega=3$ to $\Omega=50$ builds and LP-relaxes correctly, and the $\Omega=30$ MIP's root relaxation reproduces the

identical objective across two different solver configurations.

- The obstacle is resource capacity on this specific machine — 18 GB of RAM, several of which are already tied up by residual swap from earlier attempts — not a bug in the model or in the code that constructs it.
- Two distinct pressure points showed up at two different scales: outright construction memory at $\Omega=100$ (never completed, LP or MIP), and solver-internal presolve/barrier/crossover memory at just $\Omega=30$ once integrality is turned on.
- Branch-and-bound — arguably the phase most likely to need real headroom for a problem this size — has never actually run. Every failure so far happens before it starts.
- An 11-order-of-magnitude spread in the model's coefficients (5×10^{-4} to 2×10^7), visible in Gurobi's own presolve report, is a plausible contributor to Run 2's crossover instability — traceable to very large "obviously non-binding" placeholder constants (10^6 – 10^9) standing in for two not-yet-calibrated parameters.

1 More RAM.

A
machine
with
meaningfully
more
headroom
— a
lab
machine
or a
cloud
instance
— is
the
most
direct
route
to an
actual
 $\Omega=100$
result,
and
the
only
option
that
requires
no
changes
to the
formulation.

2 Shrink the placeholder coefficients.

Replace
the
 10^6 –
 10^9

"non-binding" placeholders (node-handling capacity; one arc-throughput fallback) with smaller values that are still safely non-binding. Untested, but a cheap, targeted response to the specific instability Run 2 showed.

3

Try disabling crossover.

Gurobi's `Crossover=0` is a one-line, no-model-change test of whether branch-

and-
bound
can
proceed
directly
from
the
interior-
point
solution,
skipping
the
phase
that
failed
in
Run
2.



Further construction-code rewrites.

Already
tested
and
ruled
out —
the
bulk-
insert
conversion
(\$4)
made
no
measurable
difference
at any
scale
tried.



Reducing the scenario count alone.

Not a
workaround

— per
§1, it
changes
what
the
CVaR
objective
means,
not
just
how
expensive
it is to
compute.

All figures reproduced from live runs during the 2026-09-09 working session.

Source scripts & full solver logs: `aps_usarpac/scripts/vif_phase5_structural_validation.py`,
`vif_phase5_mip_test_beta08_n30.py`, and `PHASE5_NOTES.md`.

Machine: MacBook Pro, Apple M3 Pro, 12 cores, 18 GB unified memory, macOS 24.6.0 · Gurobi 11.0.3
(academic license).